

Project Report: California House Price Predictor

Author: Sandeep Kumar **Date:** July 2026 **Live demo:** <https://sandeep-house-price.streamlit.app> **Source code:** <https://github.com/Sandeon-8104/house-price-predictor>

1. Objective

Build an end-to-end machine learning pipeline that predicts the median house price of a California district from its demographic and geographic features, and deploy it as a public interactive web application.

2. Dataset

California Housing dataset (1990 US census), loaded via scikit-learn's `fetch_california_housing()`.

- **20,640 rows** — each row is one California district
 - **8 features:** median income (MedInc), house age, average rooms, average bedrooms, population, average occupancy, latitude, longitude
 - **Target:** `MedHouseVal` — median house value in \$100,000s
-

3. Pipeline Steps

Step 1 — Data Acquisition (`src/get_data.py`)

Downloaded the dataset and saved it as `data/california_housing.csv` so all later steps work from a local file.

Step 2 — Exploratory Data Analysis (`src/explore_data.py`)

Generated statistics and four plots (histograms, income-vs-price scatter, geographic price map, correlation heatmap). Key findings:

- **No missing values.**
- **MedInc is the strongest linear predictor** of price (correlation 0.688); all other features are weak ($|r| < 0.16$).
- **Location clearly matters** — the price map shows expensive clusters around San Francisco and Los Angeles — yet Latitude/Longitude have near-zero *linear* correlation. This motivated both feature engineering and the use of non-linear models.

- **Problems found:** prices capped at exactly 5.0 (\$500k), and impossible outliers (districts with 141 average rooms, 1,243 average occupants).

Step 3 — Data Cleaning (`src/clean_data.py`)

Action	Rows removed	Reason
Dropped capped prices (MedHouseVal $\geq$ 5.0)	992	True value unknown; keeping them teaches the model to never predict above the cap
Dropped extreme outliers (AveRooms $\geq$ 15, AveBedrms $\geq$ 5, AveOccup $\geq$ 10, Population $\geq$ 10,000)	162	Dorms/prisons/data errors, not normal housing

Result: **19,486 clean rows** (5.6% removed). Raw data was kept untouched; each stage writes a new CSV (`raw` $\rightarrow$ `clean` $\rightarrow$ `features`).

Step 4 — Feature Engineering (`src/engineer_features.py`)

Five new features created from existing columns:

New feature	Formula	Correlation with price
DistToCity	<code>min(distance to SF, distance to LA)</code>	-0.452
RoomsPerPerson	<code>AveRooms / AveOccup</code>	0.321
BedrmsRatio	<code>AveBedrms / AveRooms</code>	-0.220
DistToSF, DistToLA	Euclidean distance to city center	-0.043 / -0.117

Highlight: `DistToCity` combines two nearly useless columns (Latitude $r=-0.144$, Longitude $r=-0.046$) into the **second-strongest predictor in the dataset** — domain knowledge encoded as a column.

Step 5 — Train/Test Split (`src/train.py`)

80% training (15,588 rows) / 20% test (3,898 rows), `random_state=42` for reproducibility. The test set is never seen during training, so evaluation measures real generalization, not memorization.

Step 6 — Model Training & Comparison (`src/train.py`)

Four models trained on identical data, evaluated on the identical test set:

Model	R ²	RMSE	Avg Error (MAE)
Linear Regression	0.654	0.584	\$42,825
Decision Tree	0.620	0.612	\$40,013
Random Forest (100 trees)	0.805	0.438	\$28,691
Gradient Boosting (200 trees)	0.792	0.452	\$31,107

Notable lesson: the single Decision Tree scored *worse* than Linear Regression — a live demonstration of **overfitting** (it memorizes training data but fails on unseen data). The Random Forest fixes this by averaging 100 decorrelated trees.

Step 7 — Evaluation of the Best Model (`src/evaluate_and_save.py`)

- **Predicted-vs-actual plot:** points hug the perfect-prediction line; spread grows for expensive districts (harder to predict).
- **Feature importance:** MedInc 0.416 → **DistToCity 0.143 (rank #2, our engineered feature)** → AveOccup 0.118 → all others below 0.07.
- **Sample check:** best test prediction was off by only \$3,068; worst sampled by \$51,451.

Step 8 — Model Persistence

Saved with `joblib.dump(..., compress=3)` : **135 MB** → **31 MB**, which fits under GitHub's 100 MB file limit with no accuracy loss.

Step 9 — Deployment (`app.py`)

Interactive **Streamlit** app: sliders for all inputs, city presets (SF / LA / Sacramento / Fresno), live prediction in dollars, and a district map. The engineered features are computed inside the app exactly as in training. Deployed free on **Streamlit Community Cloud**, pulling directly from the GitHub repository.

Sanity check of deployed model: a rich SF district predicts **\$396,511** vs a low-income inland district at **\$94,182** — income and location effects learned correctly.

4. Results Summary

- **Final model:** Random Forest, **R² = 0.805**, typical error ≈ **\$29,000** (~15% of the average district price).
- Accuracy improved from R² 0.654 (linear baseline) to 0.805 — driven by (a) cleaning misleading rows, (b) engineered features, (c) ensemble learning.
- The engineered `DistToCity` feature ranked **#2 of 13** in model importance.
- Fully reproducible: five scripts rebuild everything (data → model) in about one minute on a normal laptop.

5. Conclusion

This project demonstrates the complete ML workflow: acquiring data, exploring it to find problems, making defensible cleaning decisions, encoding domain knowledge as features, comparing models honestly on held-out data, and shipping the result as a public application. The main takeaways:

1. **EDA drives everything** — the capped prices, the outliers, and the location insight were all found by looking at the data before modeling.
2. **Feature engineering beats algorithm choice** — one engineered column (DistToCity) contributed more than most raw features.
3. **The train/test split is non-negotiable** — without it, the overfitting Decision Tree would have looked like the best model.
4. **Simple models are a necessary baseline** — the linear R^2 of 0.654 is what makes the forest's 0.805 meaningful.

Limitations & future work: prices are from the 1990 census (not current market values); distance is straight-line rather than travel time; hyper- parameter tuning (e.g. GridSearchCV) and XGBoost could push R^2 further; cross-validation would give more robust estimates than a single split.

6. References

1. Pace, R. Kelley and Ronald Barry (1997), "Sparse Spatial Autoregressions", *Statistics & Probability Letters*, 33, 291-297 — original paper behind the California Housing dataset.
 2. scikit-learn documentation — California Housing dataset: https://scikit-learn.org/stable/datasets/real_world.html#california-housing-dataset
 3. scikit-learn: Pedregosa et al. (2011), "Scikit-learn: Machine Learning in Python", *JMLR* 12, 2825-2830. <https://scikit-learn.org>
 4. pandas documentation: <https://pandas.pydata.org/docs/>
 5. Streamlit documentation: <https://docs.streamlit.io>
 6. Streamlit Community Cloud (deployment platform): <https://share.streamlit.io>
 7. Géron, A., *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* (O'Reilly) — the classic treatment of this exact project.
-

Appendix — Project Structure

```
house-price-predictor/
├── data/                # raw → clean → features CSVs
├── notebooks/plots/     # 6 EDA & evaluation plots
├── src/
│   ├── get_data.py      # Step 1: download dataset
│   ├── explore_data.py  # Step 2: EDA
│   ├── clean_data.py    # Step 3: cleaning
│   ├── engineer_features.py # Step 4: feature engineering
│   ├── train.py         # Steps 5-6: split + model comparison
│   └── evaluate_and_save.py # Steps 7-8: evaluation + save model
├── models/house_price_model.pkl # trained Random Forest (compressed)
├── app.py               # Step 9: Streamlit web app
├── report/REPORT.md     # this report
└── requirements.txt     # pinned dependencies
```